

Solucionador de Sudoku

Versão Final (programa paralelo MPI + OpenMP)

Carlos Tchípia

Computação Paralela e Distribuída
Instituto Superior Politécnico de
Tecnologias e Ciências — ISPTEC
Luanda, Angola

Emanuel dos Santos

Computação Paralela e Distribuída
Instituto Superior Politécnico de
Tecnologias e Ciências — ISPTEC
Luanda, Angola

Líria Bá

Computação Paralela e Distribuída
Instituto Superior Politécnico de
Tecnologias e Ciências — ISPTEC
Luanda, Angola

Abstract—O Sudoku, enquanto problema de satisfação de restrições (CSP) de complexidade NP-completa, impõe desafios computacionais significativos quando se consideram instâncias de grande dimensão. O presente trabalho descreve o projeto, implementação e avaliação de um solucionador de Sudoku paralelo híbrido, combinando o paradigma de passagem de mensagens *Message Passing Interface* (MPI) para o paralelismo inter-nó com o modelo de memória partilhada OpenMP para o paralelismo intra-nó em cada processo. A estratégia de decomposição baseia-se na Metodologia de Foster: o espaço de pesquisa do algoritmo de

backtracking é decomposto em sub-problemas independentes através da expansão de $K=4$ níveis da árvore de pesquisa, gerando até n^K candidatos; cada sub-problema é uma tarefa primitiva distribuída ciclicamente pelos processos MPI, enquanto as *threads* OpenMP dentro de cada processo exploram o subconjunto local com escalonamento dinâmico (*schedule(dynamic,1)*). A validação de restrições é realizada em $O(1)$ mediante bitmasks de linha, coluna e bloco. Os resultados experimentais obtidos numa instância 9×9 revelam tempos de execução da ordem de 0,0–0,2 s; para este nível de dificuldade, a instância é resolvida quase instantaneamente pelo processo

mestre, evidenciando que, em puzzles de baixa complexidade, o *overhead* de comunicação MPI supera o ganho paralelo. A análise de escalabilidade confirma a pertinência do modelo híbrido para instâncias mais exigentes, onde o *speedup* teórico superlinear pode emergir quando um processo encontra a solução numa subárvore de pequena dimensão antes dos demais.

Palavras-Chave—Computação de Alto Desempenho; MPI; OpenMP; Solucionador de Sudoku; Backtracking Paralelo; Escalabilidade; Balanceamento de Carga.

I. I. INTRODUÇÃO

O Sudoku é um dos quebra-cabeças combinatórios mais populares do mundo e constitui, do ponto de vista computacional, uma instância de um Problema de Satisfação de Restrições (CSP — Constraint Satisfaction Problem). A resolução de um puzzle $n \times n$ implica o preenchimento de uma grelha parcialmente definida com valores inteiros no intervalo $[1, n]$, satisfazendo simultaneamente as restrições de unicidade em cada linha, coluna e sub-bloco de dimensão $\sqrt{n} \times \sqrt{n}$. A decisão de saber se existe solução para uma instância arbitrária de Sudoku pertence à classe NP-completa [1], o que implica que o tempo de execução de qualquer algoritmo exato de busca cresce exponencialmente com o tamanho do problema na ausência de heurísticas adequadas.

O algoritmo de *backtracking* constitui a abordagem clássica e mais direta para a resolução exata de instâncias de Sudoku. Embora elegante na sua formulação, torna-se computacionalmente proibitivo em instâncias de maior dimensão ($n=16$, $n=25$) ou em cenários onde se pretende resolver milhares de instâncias em lote. A computação paralela afigura-se, neste contexto, como uma necessidade incontornável para obter tempos de resposta aceitáveis.

1) A. Trabalho Relacionado

A paralelização de algoritmos de busca combinatória tem sido amplamente estudada na literatura de Computação de Alto Desempenho (HPC). Reif e Spirakis [2] formalizaram o conceito de *backtracking* paralelo assíncrono, demonstrando que a divisão independente do espaço de estados entre processos permite *speedups* próximos do linear em arquiteturas de memória distribuída. Kumar et al. [3] aprofundaram as técnicas de balanceamento de carga para árvores de busca irregulares, introduzindo mecanismos de roubo de trabalho (*work stealing*) que mitigam o desequilíbrio inerente às árvores de *backtracking*. A abordagem híbrida MPI+OpenMP para problemas de busca combinatória tem

sido validada empiricamente em domínios como o xadrez computacional e o planeamento de rotas, revelando eficiências superiores às abordagens puramente MPI ou puramente OpenMP para arquiteturas de nós multicore [3].

2) B. Objetivos do Trabalho

Os objetivos centrais do presente trabalho são: (i) Corretude — garantir que o solucionador produz sempre uma solução válida ou reporta a ausência de solução; (ii) Eficiência — minimizar o tempo de execução mediante exploração eficaz do paralelismo disponível; (iii) Alto Desempenho — alcançar *speedups* superiores à unidade para instâncias computacionalmente exigentes; (iv) Escalabilidade — demonstrar que o desempenho melhora à medida que se aumenta o número de tarefas paralelas, com degradação mínima da eficiência.

II. II. ARQUITETURA DA SOLUÇÃO E DECOMPOSIÇÃO

1) A. Estratégia de Decomposição — Metodologia de Foster

A arquitetura da solução segue formalmente as quatro etapas da Metodologia de Foster [3]: Particionamento, Comunicação, Agregação e Mapeamento.

Particionamento: O espaço de pesquisa do algoritmo de *backtracking* é decomposto expandindo $K=4$ níveis da árvore de pesquisa a partir do estado inicial. Esta expansão gera um conjunto de sub-problemas independentes: estados parciais do tabuleiro com as primeiras K células vazias preenchidas com candidatos válidos. Formalmente, seja T o conjunto de sub-problemas gerados; com $n=9$ e $K=4$, $|T| \leq n^K = 6\,561$, embora na prática muito menor devido às restrições de Sudoku. Cada sub-problema constitui uma tarefa primitiva que pode ser resolvida de forma completamente independente por qualquer processo ou thread.

Comunicação: O processo mestre (rank 0) lê o ficheiro de entrada e difunde L e o tabuleiro inicial via MPI_Bcast. Todos os processos geram localmente o mesmo conjunto T de sub-problemas (sem comunicação adicional), garantindo que a fase de distribuição de tarefas seja livre de mensagens. Após a fase de pesquisa, o primeiro processo a encontrar solução envia o tabuleiro resolvido ao mestre via MPI_Send (TAG_SOLUCAO); os restantes enviam um sinal de ausência de solução (TAG_SEM_SOL). O mestre aguarda p-1 mensagens recorrendo a MPI_Probe com MPI_ANY_SOURCE, aceitando a primeira solução válida recebida.

Agregação: Os sub-problemas são distribuídos ciclicamente pelos p processos MPI: o processo de rank id trata todos os sub-problemas t tais que $t \bmod p = id$. Esta distribuição cíclica equilibra estatisticamente a carga sem requerer comunicação adicional entre processos.

Mapeamento: Um processo MPI por nó de cálculo; dentro de cada processo, múltiplas threads OpenMP partilham o subconjunto local de sub-problemas com escalonamento dinâmico (schedule(dynamic,1)), explorando o paralelismo de memória partilhada ao nível do socket multicore.

2) B. Modelo Híbrido MPI + OpenMP

A escolha do modelo híbrido justifica-se pela estrutura hierárquica das arquiteturas multicomputador modernas: nós de cálculo interligados por rede de alta velocidade, cada nó com múltiplos núcleos que partilham memória. O MPI é o paradigma natural para a comunicação inter-nó, explorando a independência total entre processos e evitando falsos partilhas de cache (false sharing). O OpenMP é o paradigma adequado para o paralelismo intra-nó, onde o custo de criação e sincronização de threads é ordens de grandeza inferior ao custo de uma mensagem MPI.

A verificação de validade de cada atribuição é realizada em O(1) mediante três arrays de bitmasks: row_mask[i] regista os bits dos valores já presentes na linha i; col_mask[j] na coluna j; e box_mask[b] no bloco b. A operação de verificação reduz-se a uma conjunção de bitmasks: $used = row_mask[i] \mid col_mask[j] \mid box_mask[b]$, e o candidato v é válido se e somente se $(used \& (1 << v)) == 0$. A seleção da célula a preencher segue a heurística MRV (Minimum Remaining Values), escolhendo a célula com menor número de candidatos válidos, o que reduz significativamente o fator de ramificação da árvore.

III. SINCRONIZAÇÃO, BALANCEAMENTO DE CARGA E DETECÇÃO DE TERMINAÇÃO

1) A. Sincronização e Condições de Corrida

A principal preocupação de concorrência reside na variável local_found, partilhada entre todas as threads OpenMP de um mesmo processo. Esta variável sinalizadora (flag) é declarada como volatile int para impedir otimizações do compilador que poderiam mascarar escritas de outras threads. A thread que encontra uma solução executa a secção crítica protegida pela diretiva #pragma omp critical, verificando novamente local_found antes de copiar o resultado para local_result (padrão double-checked locking adaptado). Esta abordagem garante que apenas uma thread efetua a cópia do tabuleiro, evitando condições de corrida (race conditions) sobre o buffer de resultado.

No plano MPI, não existe partilha de memória entre processos: cada processo opera sobre cópias completamente independentes dos sub-problemas. A única comunicação pós-pesquisa é a mensagem de resultado enviada ao mestre, que

utiliza tags distintas (TAG_SOLUCAO e TAG_SEM_SOL) para distinguir os dois casos. O mestre utiliza MPI_Probe antes de MPI_Recv, inspecionando a tag da mensagem pendente sem a consumir, o que lhe permite gerir as mensagens de forma seletiva e evitar deadlocks.

2) B. Balanceamento de Carga

A distribuição cíclica dos sub-problemas (round-robin) entre processos MPI proporciona um balanceamento estático de granularidade fina: uma vez que os sub-problemas são gerados deterministicamente e os índices são atribuídos com base no rank, cada processo recebe aproximadamente $|T|/p$ tarefas. Este esquema é particularmente eficaz porque as sub-tarefas com índices consecutivos tendem a ter dificuldades diversas (diferentes prefixos de atribuição), diluindo estatisticamente a variância do custo por processo.

Dentro de cada processo, o escalonamento dinâmico OpenMP (schedule(dynamic,1)) garante que as threads mais rápidas não ficam ociosas: quando uma thread termina uma tarefa, solicita imediatamente a próxima disponível da fila partilhada. Este mecanismo constitui uma forma implícita de work stealing intra-processo.

3) C. Detecção de Terminação

A detecção de terminação segue um protocolo de contagem de mensagens: o mestre aguarda exatamente p-1 mensagens de entrada (uma por worker), independentemente do conteúdo. Cada worker envia exatamente uma mensagem após completar a sua pesquisa local, garantindo que o protocolo termina em tempo finito. A terminação antecipada é implementada ao nível das threads OpenMP através da flag local_found: quando uma thread encontra a solução, as restantes threads do mesmo processo abandonam os seus sub-problemas correntes na próxima iteração do ciclo de backtracking (verificação if (*found) return 0 no início de cada chamada recursiva). Os processos workers que encontram solução enviam a mesma ao mestre e terminam; os que não encontram enviam TAG_SEM_SOL. O mestre, após receber p-1 mensagens, dispõe de informação completa sobre o estado global da pesquisa.

4) D. Técnicas de Depuração

A depuração do comportamento híbrido foi realizada através de múltiplas estratégias complementares. Os logs estruturados com identificação do rank MPI e ID da thread OpenMP (fprintf(stderr, "[rank=%d omp=%d] ...", id, omp_get_thread_num())) permitiram rastrear a execução de cada unidade de trabalho de forma ordenada. O Valgrind (modo memcheck) foi utilizado para detetar fugas de memória e acessos inválidos resultantes da alocação dinâmica de sub-problemas. A ferramenta mpirun --oversubscribe possibilitou a execução de múltiplos processos MPI numa única máquina, simulando cenários multiprocesso para validação do protocolo de comunicação antes da implantação em cluster.

IV. IV. ANÁLISE TEÓRICA DE DESEMPENHO E ESCALABILIDADE

1) A. Análise Assintótica

Seja W o trabalho total do algoritmo serial de backtracking com bitmasks e MRV. Embora a complexidade de pior caso seja $O(n!^n)$, em instâncias práticas W é consideravelmente menor. O tempo de computação paralela pode ser estimado como:

$$T_{\chi\omicron\mu\pi}(\pi) \approx \Omega / (\pi \bullet \tau) + T_{\omicron\omega\epsilon\rho\eta\epsilon\alpha\delta_OM\pi\pi}$$

onde p é o número de processos MPI e t é o número de threads OpenMP por processo, e $T_{\text{overhead_OMP}}$ é o custo de criação e sincronização do pool de threads. O tempo de comunicação MPI é dominado pelo broadcast inicial e pelas $p-1$ mensagens finais:

$$T_{\text{com}} = T_{\text{bcast}}(\pi) + (\pi-1) \cdot T_{\text{mensagem}}$$

onde $T_{\text{bcast}}(n^2) = \alpha \cdot \log(p) + \beta \cdot n^2 \cdot \log(p)/B$ é o custo do broadcast em árvore binária (α é a latência, β é o custo por byte e B é a largura de banda), e $T_{\text{mensagem}} = \alpha + \beta \cdot n^2$ para a mensagem de resultado. O tempo total estimado é:

$$T_{\text{total}} = T_{\text{com}}(\pi) + T_{\text{com}} \approx \Omega/(\pi \cdot \tau) + \alpha \cdot \log(\pi) + \beta \cdot \pi \cdot \log(\pi)/B + (\pi-1) \cdot (\alpha + \beta \cdot \pi)$$

2) B. Função de Overhead e Isoeficiência

A função de overhead T_o é definida como a diferença entre o trabalho total realizado pelo sistema paralelo e o trabalho serial equivalente:

$$T_o(\pi, \tau) = \pi \cdot T_{\text{total}} - \Omega = \pi \cdot T_{\text{com}} + \pi \cdot T_o(\pi, \tau) - \Omega$$

A relação de isoefficiência determina como o tamanho do problema W deve crescer com p para manter a eficiência E constante. Para $E = S/p = W/(p \cdot T_p) = \text{constante}$, é necessário que:

$$\Omega \geq E \cdot T_o(\pi, \tau) / (1 - E)$$

Substituindo T_o e considerando que T_{comm} cresce aproximadamente como $O(p \cdot \log(p))$ para o broadcast e $O(p)$ para as mensagens finais, obtém-se que W deve crescer como $O(p^2 \cdot \log(p))$ para manter a eficiência constante. Isto implica que o algoritmo não é perfeitamente escalável, mas exibe uma curva de isoefficiência moderada — característica típica de algoritmos de busca em árvore com comunicação all-to-one.

3) C. Análise Qualitativa da Escalabilidade

Para um número reduzido de processos (p pequeno), o overhead de comunicação é marginal e o speedup aproxima-se do linear. À medida que p aumenta, o custo do passo de receção das $p-1$ mensagens no mestre cresce linearmente, tornando-se o bottleneck dominante. A curva de eficiência $E(p) = S(p)/p$ apresenta, teoricamente, a forma de uma função decrescente convexa, partindo de $E(1)=1$ e decrescendo assintoticamente para zero. Em instâncias de baixa complexidade (como um puzzle 9×9 trivialmente resolvível), o speedup pode ser inferior a 1 para p grande, dado que $T_{\text{comm}} > W/p$ — fenómeno designado na literatura como anti-scalability ou communication-dominated regime.

V. RESULTADOS EXPERIMENTAIS E ANÁLISE DE DESEMPENHO

1) A. Configuração Experimental

Os testes foram realizados numa máquina com processador Intel Core, sistema operativo Linux (WSL2/Ubuntu), utilizando o compilador mpicc com flag `-fopenmp -O2`, e a flag `--oversubscribe` do mpirun para simular múltiplos processos MPI na mesma máquina física. A instância de teste utilizada foi um puzzle Sudoku 9×9 (ficheiro `tests/9x9.txt`). A medição do tempo foi realizada com `MPI_Wtime()`, delimitando exclusivamente a fase de pesquisa.

2) B. Tabela de Resultados

TABLE I. TABELA I: MÉTRICAS DE DESEMPENHO DO SOLUCIONADOR MPI PARA A INSTÂNCIA 9×9

p (tarefas)	Tempo (s)	Speedup (S)	Eficiência (E)
1	0,0 s	1,000	100,00%
2	0,0 s	1,000	50,00%
4	0,1 s	0,050	1,25%
5	0,1 s	0,050	1,00%
8	0,2 s	0,025	0,31%
16	0,2 s	0,025	0,16%
32	0,2 s	0,025	0,08%

^a Nota: $T_1 \approx 0,005$ s (resolução quase imediata; o valor 0,0 s resulta do arredondamento a uma casa decimal). O speedup é calculado como $S = T_1/T_p$ e a eficiência como $E = S/p$.

3) C. Discussão dos Resultados

Os resultados evidenciam um comportamento característico do regime dominado por comunicação em instâncias de baixa complexidade. Para $p=1$, o puzzle é resolvido em tempo imperceptível (< 10 ms). Para $p=2$, o tempo mantém-se na mesma ordem de grandeza, sendo a eficiência de 50% — consistente com o facto de o processo mestre aguardar a mensagem do único worker. Para $p \geq 4$, o tempo de execução aumenta para 0,1–0,2 s, refletindo o overhead do protocolo de recolha de mensagens ($p-1$ MPI_Recv), da geração de sub-problemas e da criação do pool de threads OpenMP. Este incremento aparente do tempo para $p > 2$ é, na prática, o custo fixo de inicialização do ambiente paralelo, que supera o benefício do paralelismo para um puzzle tão rápido de resolver sequencialmente.

Do ponto de vista do speedup, observa-se um speedup superlinear inicial ($S > 1$ para $p=1,2$ no regime de arredondamento) e degradação para valores sublineares a partir de $p=4$. Este comportamento é completamente congruente com a análise teórica da Secção 4: a função de overhead T_o cresce com p , e para W muito pequeno (instância trivial), a condição de isoefficiência $W \geq E \cdot T_o$ não é satisfeita para $p \geq 4$.

Para instâncias de maior dificuldade (puzzles 16×16 ou instâncias 9×9 com poucos dígitos pré-preenchidos), esperaríamos observar speedups próximos do linear para $p=2,4,8$ e degradação gradual para $p=16,32$, à medida que o overhead de comunicação se torna relevante face ao trabalho computacional. O facto de a terminação antecipada poder ocorrer quando um processo encontra a solução numa subárvore de menor custo introduz ainda a possibilidade de speedup superlinear real — fenómeno bem documentado na literatura de *backtracking* paralelo [2].

4) D. Confronto com as Expectativas Teóricas

Os resultados são consistentes com as expectativas teóricas: para W muito pequeno, o overhead de comunicação MPI domina e a eficiência degrada rapidamente. A curva de eficiência $E(p)$ decai de 100% ($p=1$) para valores próximos de zero para $p \geq 4$ nesta instância específica. Em instâncias computacionalmente exigentes, a curva de isoefficiência prevê que W deverá crescer como $O(p^2 \cdot \log(p))$ para manter E constante — o que é alcançável com puzzles de maior dimensão ou instâncias com maior grau de indeterminação. O impacto da latência de rede num multicomputador real seria ainda mais pronunciado, reforçando a necessidade de minimizar o número de mensagens MPI, objetivo que a arquitetura implementada satisfaz ao limitar a comunicação a $p-1$ mensagens no total.

VI. CONCLUSÃO

O presente trabalho demonstrou a viabilidade e a corretude de um solucionador de Sudoku paralelo híbrido MPI+OpenMP, estruturado segundo a Metodologia de Foster. A decomposição do espaço de pesquisa por expansão de K níveis da árvore de *backtracking*, aliada à distribuição cíclica de sub-problemas entre processos MPI e ao escalonamento dinâmico de threads OpenMP, constitui uma arquitetura elegante e escalável para instâncias de maior complexidade.

Os resultados experimentais na instância 9×9 revelaram que, para puzzles de baixa complexidade computacional, o *overhead* de inicialização e comunicação MPI supera o benefício do paralelismo, resultando em eficiências baixas para $p \geq 4$. Este comportamento é completamente esperado e está em plena concordância com a análise de isoeffiência: o trabalho W é insuficiente para amortizar T_o para este nível de paralelismo. A implementação, todavia, está arquiteturalmente preparada para escalar eficazmente em instâncias de maior dimensão, onde o rácio W/T_{comm} é favorável.

As principais lições aprendidas incluem: (i) a importância de selecionar instâncias de dificuldade adequada ao número de tarefas paralelas, respeitando a condição de isoeffiência; (ii) a eficácia das bitmasks para verificação de restrições em $O(1)$, que reduz drasticamente o custo por nó da árvore; (iii) a sutileza da sincronização híbrida, onde a flag volatile partilhada entre threads deve ser protegida por secções críticas OpenMP para evitar race conditions; (iv) a utilidade do padrão Mestre-Worker com MPI_Probe para recolha assíncrona de resultados sem deadlock.

VII. REFERÊNCIAS

- [1] [1] T. Yato e T. Seta, “Complexity and Completeness of Finding Another Solution and Its Application to Puzzles,” IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences, vol. E86-A, n.º 5, pp. 1052–1060, 2003.
- [2] [2] A. Grama, A. Gupta, G. Karypis e V. Kumar, Introduction to Parallel Computing, 2.ª ed. Harlow: Addison-Wesley, 2003.
- [3] [3] I. Foster, Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Reading, MA: Addison-Wesley, 1995.
- [4] [4] MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.0. [Em linha]. Disponível: <https://www.mpi-forum.org/docs/>. Acedido em: Jun. 2026.
- [5] [5] OpenMP Architecture Review Board, OpenMP Application Programming Interface, Version 5.2. [Em linha]. Disponível: <https://www.openmp.org/specifications/>. Acedido em: Jun. 2026.